United Nations
Office of Information and Communications Technology

GOOGLE SUMMER OF CODE
2026
PROPOSAL

Project Title: OG–CLEWS: Integrating Open-Source Economic and Environmental Models for Sustainable Development (Contributor 1)

Organization: United Nations Office of Information and Communication Technology
Mentors: Mr. Marcelo LaFleur (lafleurm@un.org)
Mr. Alfonso Acosta Gonçalves (acostagoncalves@un.org)

1. Personal Information

Full Name: Tejas Singh Bhati
Email: tejassinghbhati077@gmail.com
Github: https://github.com/tejassinghbhati
LinkedIn: https://www.linkedin.com/in/tejassinghbhati/
Phone Number: +91 96643 33299
Discord: tejassinghbhati_74330
University: Indian Institute of Information Technology, Sonepat, India
Location: Sonepat, Haryana, India
Year Of Study: 2024-2028, Second year
Country/ Residence: India
TimeZone: IST Asia/Kolkata | UTC +5:30
Primary Language: I am proficient in English for communication, both spoken and written.
Preparation Project: https://github.com/tejassinghbhati/ogclews-prep

About Me

I am a second year student pursuing a Bachelor's in Technology - Computer Science (Data Science and Analytics) from Indian Institute of Information Technology, Sonepat. My semester exams will get over by late April, I have 3 months of available vacation period after that. This allows me to have enough time to devote to the project. If I am selected I shall be able to work around 40-50 hours on the project every week. I have spent a significant amount of time before this application not just gaining knowledge about OG-Core and CLEWS, yet also researching through the systems and contributing to the repo. The proposal ahead consists of all that information and experience I gained and my readiness for this project.

---

2. Contributions

I have always been an enthusiast for data science and scientific programming. I have spent my college years exploring and gaining experience in various tech domains, doing work in open source, creating models and developing softwares which can have an impact on the world. This has helped me enhance my capacity to work across scientific softwares.

Before contributing, I invested time in understanding the repository's setup pipeline, execution flow, and environment configuration logic across different operating systems. This included reproducing reported issues tracing setup-state side effects, and analysis how platform-specific scripts interact with the broader execution workflow. With all the issues I raised, I made sure to upload appropriate evidence and proper reproduction steps.
I have tried to be as consistent as I can in raising issues and pull requests on the MUIOGO repository.

Below are all of my Merged Pull Requests (11):

1. fix: remove broken /myfunc placeholder route for the issue [Bug] /myfunc route crashes on every call - wrong number of arguments passed to Download()
    - Identified a dead route that was never implemented but remained callable, causing crashes on every invocation
    - Cleaned up the API surface by removing the stub entirely rather than patching around it.

2. fix: return jsonify response in CopyCase success path for the issue [Bug]/copyCase return 500 Internal Server Error on success - missing jsonify()
    - Diagnosed a subtle server error where the success branch of a route returned a plain dict instead of a proper Flask JSON response, causing a 500 despite the operation succeeding
    - Fixed the response contract so clients receive consistent, parseable output on both success and failure paths

3. fix: guard against None value in OsemosysClass PARAM dict build for the issue [Bug] AttributeError crash in OsemosysClass._init_ when a parameter value is None
    - Traced an AttributeError crash to a missing null-check during parameter dictionary construction at initialization time
    - Added a defensive guard so the class handles incomplete or partially populated parameter configs without crashing the entire solver setup

4. fix: remove duplicate ‘RYTM’ key from DEFAULT_F, UPDATE_F and GEN_F for the issue [Bug] Duplicate key “RYTM” in DEFAULT_F, UPDATE_F and GEN_F causes silent data loss in Config.py
    - Found a silent data loss bug caused by a duplicate dictionary key — Python silently overwrites earlier values, meaning one configuration was always being ignored
    - Removed the duplicate and flagged the pattern across all three related config structures

5. fix: guard casename against None before filesystem ops in deleteCaseRun for the issue [Bug] deleteCaseRun performs filesystem operations before validating casename is not None
    - Identified a security/stability risk where filesystem delete operations were being executed before validating that the case name was not None
    - Moved validation ahead of any I/O, following the fail-fast principle to prevent unintended file system side effects

6. fix: use context managers to prevent file handle leaks in FileClass for the issue [Bug] File handles are not closed when an exception occurs in FileClass.py
    - Spotted resource leak potential where file handles weren't being closed if exceptions occurred mid-operation
    - Refactored to use ‘with’ statements (context managers) across all file operations, ensuring handles are always released regardless of success or failure

7. fix: validate case existence in /setSession for the issue [Bug] /setSession allows arbitrary session hijacking - bypasses ownership guards
    - Discovered that the /setSession route accepted arbitrary case names without verifying they existed, enabling session hijacking by switching into another user's case
    - Added an existence check and ownership guard before allowing session assignment

8. fix: add session ownership check before deleteCase and copyCase (#48) for the issue [Bug] /deleteCase and /copyCase perform destructive filesystem operations without validating session ownership
    - Found that destructive filesystem operations (delete, copy) had no verification that the requesting session actually owned the target case
    - Introduced an ownership validation layer before any write or delete is permitted, closing a significant access control gap

9. fix: propagate exceptions from CustomThread to caller on join() (#45) for the issue [Bug] CustomThread silently discards exceptions from background solver threads - callers receive None on failure
    - Identified that background solver thread failures were being silently swallowed — callers always received None whether the solver succeeded or crashed
    - Reworked the threading wrapper to capture and re-raise exceptions on join(), making failures observable and debuggable by the caller

10. fix: resolve solver paths via env var / PATH / bundled fallback (#43) for the issue [Bug] Non-Windows solver paths in OsemosysClass are hardcoded dead-end stubs
    - Found that non-Windows solver paths were hardcoded placeholder stubs that would never resolve on Linux or macOS
    - Implemented a three-tier resolution strategy: check environment variable → search system PATH → fall back to a bundled binary, making the solver portable across platforms

11. fix: replace hardcoded Windows path separators with os.sep(#34) for the issue [Task] Refactor hardcoded Windows path separators in UploadRoute and SyncS3 to use os.sep
    - Identified Windows-specific \\ separators hardcoded in upload and S3 sync routes, causing silent path failures on Unix systems
    - Replaced all occurrences with os.sep to make path handling cross-platform without any behavior change on Windows

Other Pending Pull Requests include:

1. Fix/295 casename validation for the issue [Enhancement] Add Case Name Validation to Reject Path-Unsafe Characters
    - Identified that case names were accepted without sanitization, allowing characters like ../, /, and \ that could be used to construct path traversal attacks against the filesystem
    - Implementing a validation layer that rejects path-unsafe characters at the input boundary before any downstream filesystem or session operation is performed

2. Fix: #278: resolve fatal UI TypeError crash when deleting commodity due to undefined model.comms typo for the issue [Bug] UI Crash (TypeError) when Deleting Commodities with Out-of-Sync Indices in AddCase.js

    - Traced a fatal frontend crash to a typo in a JavaScript property reference (model.comms instead of the correct property), which caused the index lookup to fail silently and then throw a TypeError on deletion
    - Fixing the property reference and adding an index bounds check so the delete operation degrades gracefully rather than crashing the entire UI component

3. fix: add dedicated uninstall scripts to reverse local setup state for the issue [Task] Add uninstaller script for setup-created local state
    - Identified that the existing setup scripts had no corresponding teardown path, meaning developers who ran setup had no clean way to reverse environment changes during debugging or re-installation
    - Building dedicated uninstall scripts that mirror the setup logic in reverse, ensuring reproducible local environment management across platforms

I tried to contribute in a way where I kept my focus on improving reproducibility of local environmental setup. The experience of working on the repository helped me grow a lot personally myself, I understood the importance of communication with maintainers and the philosophy in which open source projects are designed.

---

3. Why Am I Here?

3.1 Inspiration

I was 8 years old when the Paris Agreement took place. I do not remember how 195 parties came together to work on building a more sustainable future, yet I remember being scared of the fact that Climate Change is such a looming crisis on our planet. I used to wonder curiously about the future of our planet, ‘Why isn't anybody doing anything?’ I remember asking that to my parents. 11 years have passed, if my nine-year-old self would see me contributing to this project, I don’t know how he would react, but I think he would finally feel like someone is doing something.

3.2 Understanding the models

I started with researching OG-Core, OG stands for Overlapping-Generations, it models core theory, logic, and solution method algorithms that allow for dynamic general equilibrium analysis of fiscal policy. The model output includes changes in macroeconomic aggregates (GDP, investment, consumption), wages, interest rates, and the stream of tax revenues over time.

On the other hand, OSeMOSYS is the Open Source energy MOdeling SYStem which is designed to fill the gap in the analytical toolbox available to the energy research community and energy planners in developing countries. I love the fact that it requires no upfront financial investment and hence makes such an amazing energy tool available to a community of students, business analysts, government specialists and developing country energy researchers like me.

The CLEWS(Climate, Land, Energy, Water Systems) framework is built on OSeMOSYS, it maps the interactions, synergies, and trade-offs among land use, the energy sector, and water systems under climate change scenarios.

3.3 The Gap

Both of them are undoubtedly very powerful tools, they are mature and widely used in over 20 countries around the world. These models have supported developing countries facing challenges in achieving sustainable development because they are open source, assumptions are visible and results are reproducible, supporting informed policy discussion. Yet both of these models operate as separate tools.

3.4 The Integration

OG-Core speaks the language of pure economics

OG-Core operates entirely on economic value and price signals, and it does not model any physical or natural resource constraints. By that statement I mean that the model ensures that total monetary output equals total monetary expenditure (Consumption + Investment + Government spending + Net Exports). It does not track tons of materials, joules of energy, or hectares of land.

The inputs to production are grouped into abstract, monetary valued aggregates, also the model balances supply and demand entirely through prices. It assumes that as long as prices are clear, the economy functions and henceforth ignores whether there are actually enough physical resources (like water, minerals or energy) in the real world to physically produce the output.

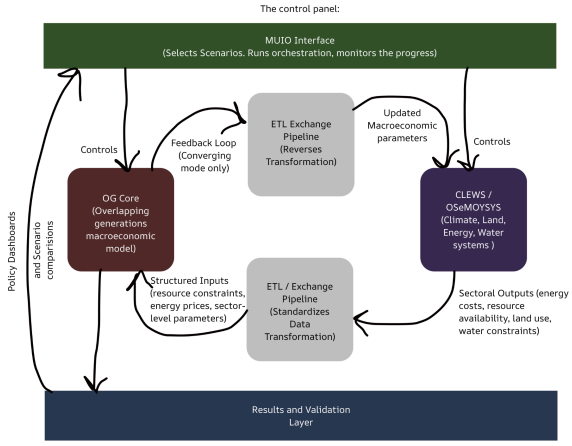
So in the case where you need to analyze physical constraints (like climate impacts, energy limits, or material bottlenecks), OG-Core would need to be soft-linked to a biological or integrated assessment model like CLEWs, as OG-Core itself is blind to physical realities.

CLEWs speaks the language of physics and engineering

CLEWS uses plain text data files for its inputs. At its core CLEWS is governed by the physical laws of conservation since the model strictly enforces that whatever goes in must come out, be stored or be transformed. Also CLEWS doesn't just assume production increases if demand or prices rise, rather it checks if the physical capacity exists.

Hence, CLEWS is what modelers call a "bottom-up" systems engineering framework. It builds system representation from individual, physically defined technologies (pipelines, solar panels, tractors, crop types, dams) and links them together to meet a demand, rather than top-down equations based on economic behavior or historical price elasticities.

Our project, the OG-CLEWS framework will create a standardized, automated interface between them and a shared execution and visualization layer, enabling integrated analyses that are not currently possible with existing tools.



It is fascinating to even imagine how much this would provide to policymakers. An easy-to-use, accessible tool to check their plans and strategies and back them with an evidence-based, transparent assessment tool. They will gain a more holistic view of their policies, encompassing natural resources management, the energy sector, and their macroeconomic and fiscal implications, in a novel approach that is not currently available to them.

Figure: Proposed OG-CLEWS integrated workflow

3.5 The Translation Problem

When we speak of translation, we mean to map a physical scarcity or physical cost output by CLEWs into an economic shock or policy lever inside OG-Core. Now since OG-Core has no concept of natural resources, the physical constraints must be expressed as adjustments to the production functions, prices or capital stocks that the overlapping generations model understands.

Some specific translational issues to solve:

Water availability in CLEWS - what parameter in OG-Core?
CLEWs might show a severe deficit due to drought, forcing agriculture yields down and reducing power plant cooling. OG-Core does not have a "water" input. If we translate this physical debt into shock to the Total Factor Productivity (Z or Z_annual), believing 'capital and labour are now less efficient at creating output' but then what will be the parameter to convert a 10% water deficit into a X% drop in aggregate TFP?

Energy Systems Transitions in CLEWS - Capital depreciation rate in OG-Core?
Now since CLEWs require retiring fossil fuel plants before the end of their useful life and rapidly building renewables to achieve climate targets, the question is how do we translate this for OG-Core? Perhaps a logical transmission channel would be an increase in annual capital depreciation rate parameter (Δ_annual). However, does forcing Δ higher uniformly across the entire capital stock accurately reflect the cost of energy transitions, or does it require separating energy capital from general physical capital? If we use multi-industry capabilities, how exactly do we apply unequal deprecation offsets?

CO2 Emission Caps in CLEWS - Carbon tax policy in OG-Core?
CLEWS can calculate the 'shadow price' of carbon which is the economic penalty required to force the energy model to stay under emissions cap. On the other hand OG-Core evaluates fiscal policy. We could map the shadow price to a consumption tax (tau_c) on carbon-intensive goods or a specific business tax (cit_rate adjustment). But this would accidentally make the government's balance sheet look artificially heavy because there is no way to model this 'tax' without generating government revenue

3.6 Readiness

Integrating a biophysical model with a macroeconomic model is not a simple data engineering task; it is a difficult theoretical economics challenge. There are a few noticeable points when we consider this:

1. **No 1-to-1 mapping:** We cannot just plug a petajoule into a Python function and get a tax rate out. It requires defining economic elasticities.
2. **Risk of Double Counting**: If we translate an energy transition cost into both a drop in TFP and an increase in capital deprecation, we will double-count the economic damage and break the equilibrium.
3. **Accounting Trap:** As noticed in the carbon trap proxy, translating physical costs into tax equivalents breaks the macroeconomic accounting identities if the 'fake revenue' is not appropriately recycled or burned.

Henceforth, writing code for the ETL data pipeline (transformer.py) is actually the second step of this project. While the first step is rigorous framework design. I believe this part has the highest significance as of now and I look forward to delivering its execution. There is no project in the GSoC cycle I could be more committed to delivering.

Some Evidences that I gathered:

```python
def resource_constraint(Y, C, G, I_d, I_g, net_capital_flows, RM):
    r"""
    Compute the error in the resource constraint.

    .. math::
      \begin{split}
        \text{rc_error} &= \hat{Y}_t - \hat{C}_t -
      \Bigl(e^{g_y}\bigl[1 + \tilde{g}_{n,t+1}\bigr]\hat{K}^d_{t+1} -
      \hat{K}^d_t\Bigr) - \delta\hat{K}_t - \hat{G}_t - \hat{I}_{g,t} ... \\
        &\qquad -\: \hat{\text{net capital outflows}}_t + \hat{RM}_t
      \end{split}

    Args:
        Y (array_like): aggregate output by industry
        C (array_like): aggregate consumption by industry
        G (array_like): aggregate government spending by industry
        I_d (array_like): aggregate private investment from domestic households
        I_g (array_like): investment in government capital
        net_capital_flows (array_like): net capital outflows
        RM (array_like): aggregate remittances

    Returns:
        rc_error (array_like): error in the resource constraint

    """
    rc_error = Y - C - I_d - I_g - G - net_capital_flows + RM

    return rc_error
```

Figure 1: The core resource constraint function in aggregates.py. This identity (Y = C + I + G + NX) demonstrates why CLEWs physical demands cannot be naively injected; any external shock must allow this endogenous equation to re-clear, otherwise the transition path iteration (TPI) will fail to converge.

```python
def constraint_checker_TPI(b_dist, n_dist, c_dist, t, ltilde):
        b_dist (Numpy array): distribution of capital at time t
        n_dist (Numpy array): distribution of labor at time t
        c_dist (Numpy array): distribution of consumption at time t
        t (int): time period
        ltilde (scalar): upper bound of household labor supply

    Returns:
        None

    Raises:
        Warnings: if constraints are violated, warnings printed

    """
    if (b_dist <= 0).any():
        logging.info(
            f"WARNING: Aggregate capital is less than or equal to zero in period {t}."
        )
    if (n_dist < 0).any():
        logging.info(
            f"WARNING: Labor supply violates nonnegativity constraints in period {t}."
        )
    if (n_dist > ltilde).any():
        logging.info(
            "\tWARNING: Labor suppy violates the ltilde constraint",
            " in period %.f." % t,
        )
    if (c_dist < 0).any():
        logging.info(
            "\tWARNING: Consumption violates nonnegativity",
            " constraints in period %.f." % t,
        )
```

Figure 2: The constraint_checker_TPI function in household.py. A key goal is to ensure that physical constraints passed from CLEWS do not inadvertently force OG-Core's optimizing households into mathematically impossible states (eg. violating non negativity constraints on consumption or labor hours).

```python
def run_TPI(p, client=None):

    # Restore the schema objects (they're not needed by workers anyway)
    for attr, value in schema_backup.items():
        try:
            setattr(p, attr, value)
        except:
            pass

    # TPI loop
    while (TPIiter < p.maxiter) and (TPIdist >= p.mindist_TPI):
        outer_loop_vars = (r_p, r, w, p_m, BQ, RM, TR, theta)
        # compute composite good price
        p_i = (
            np.tile(p.io_matrix.reshape(1, p.I, p.M), (p.T + p.S, 1, 1))
            * np.tile(p_m.reshape(p.T + p.S, 1, p.M), (1, p.I, 1))
        ).sum(axis=2)
        p_tilde = aggr.get_ptilde(p_i[:, :], p.tau_c[:, :], p.alpha_c, "TPI")

        # Initialize Euler errors
        euler_errors = np.zeros((p.T, 2 * p.S, p.J))
        # Solve for household decisions
        results = []
        if client:
            futures = []
            for j in range(p.J):
                guesses = (guesses_b[:, :, j], guesses_n[:, :, j])
                f = client.submit(
                    inner_loop,
```

Figure 3: The minimal script simply passes endogenous GDP from OG-Core to static demand in CLEWS, and passes physical deficits from CLEWS back as TPF penalty to OG-Core, looping until the equilibrium is reached.

Sources: [OG-Core Repository]

[OSeMOSYS Repository]

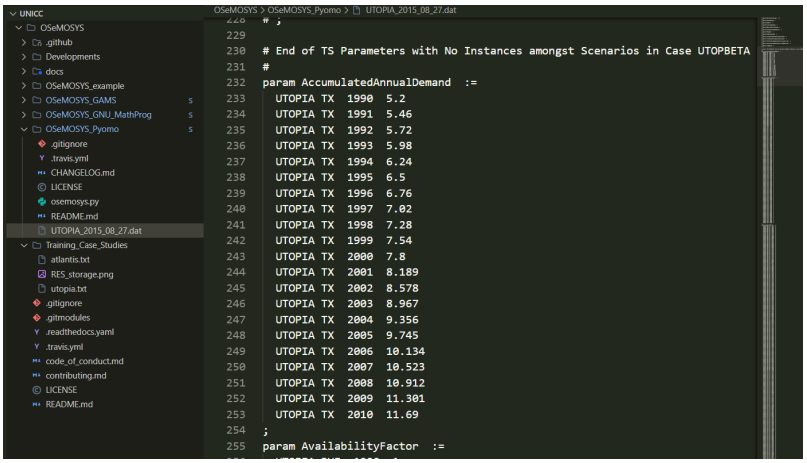


Figure 4: Exogenous demand in CLEWS. Notice that in the OSeMOSYS parameter file (UTOPIA_2015_08_27.dat), systemic demand is passed as a hardcoded static matrix. In OG-Core, this exact same metric is an endogenous output of household utility maximization. A blind software link here would mathematically crash the solver by over-constraining the equation.

---

4. Framework Before Implementation

Are the assumptions of OG-Core and CLEWS compatible at the points where we propose to connect them?

4.1 Summary of Theoretical Compatibility Challenges

Connecting the bottom up physical optimization model (CLEWS) and top down macroeconomic general equilibrium model (OG-Core) presents fundamental theoretical frictions. In the previous section we have raised enough considerations to understand that there is no direct 1-to-1 mapping between these two divergent paradigms. Naive data handoffs lead to severe double-counting, violation of macroeconomic accounting identities, and inconsistent price-quality equilibriums.

Conclusion to model linkage

CLEWs provides physical realism that OG-Core lacks, and OG-Core dictates the endogenous macroeconomic demand that CLEWs assumes is fixed. I believe that OG-Core and CLEWS are incompatible in a static, one-way pass. However, they can become highly synergistic when deployed in an iterative, ‘soft-linking’ convergence loop.

The gap can be bridged by carefully defining economic transmission channels(translating physical shocks into Total Factor Productivity, dynamic capital deprecation offsets, and specific tax rate proxies).

4.2 Suitable Framework for Execution

The primary objective of this project is to conduct a rigorous mathematical and philosophical assessment of both models’ underlying architectures. The goal is to identify viable, non

distorting connection points and develop the fundamental theoretical framework necessary for any future software integration layer.
Therefore here I have discussed the framework of execution and how I would try to approach it:

Phase 1: Philosophical and Mathematical Assessment
I would conduct a deeper dive into the objective functions of both the models. Allowing me to evaluate where exogenous variables in CLEWs (like gdp or population) conflict with endogenous variables in OG-Core.
Here the goal would be to produce a carefully curated assessment document identifying every single 'hard stop' that is, every place where the models mathematically contradict each other.

Phase 2: Mapping and Evaluating Link Typologies
Instead of assuming how we link them, we would test hypotheses. I will map out potential translation points, for instance trying to map water scarcity to TFP vs. mapping it to a sector specific output constraint and evaluate the economic distortions each creates.

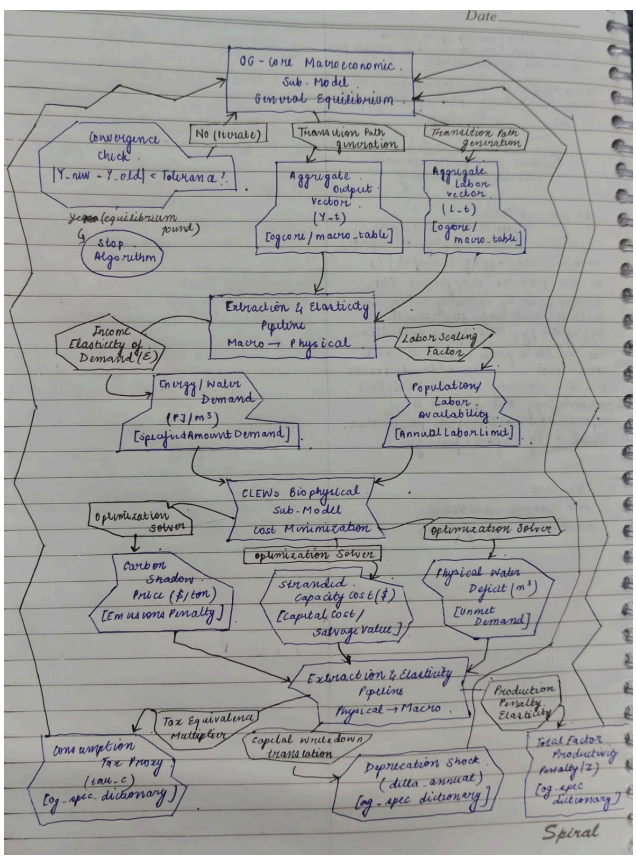


Figure: This diagram illustrates the iterative soft-linking convergence loop between the top-down macroeconomic model (OG-Core) and the bottom-up physical optimization model (CLEWs). It explicitly details the variable names and mathematical parameters being exchanged.

And here the goal would be to develop a 'Translation matrix' that doesn't just link connections but also explains the theoretical justification and risks of each connection.

Phase 3: Prototyping and Integration Framework
I would build a minimalist, proof-of-concept script that executes one agreed-upon soft link for instance passing a single carbon copy to test if the model can successfully solve without its resource constraints crashing.
The goal would be to deliver the validated foundational framework that in the future an automated pipeline can be built confidently during development phases.

This table represents the theoretical core of the 'soft-linking' process. It maps exactly how physical outputs from the CLEWs biophysical optimization solver are translated into the macroeconomic parameters (og_spec) required by OG-Core.

Phase/ Sector	CLEWs Physical Output	OG-Core Parameter Target	og_spec variable	Theoretical Justification	Primary Risk/ Mathematical Friction
CLIMATE	Carbon Shadow Price to meet emissions cap	Consumption Tax Proxy	tau_c (Txl array)	A shadow price on carbon acts as a penalty on consumption of emission-intensive goods. Raising tau_c mathematically mimics penalty	Revenue Recycling Trap: tau_c generates fake tax revenues for the government. Must build rules to artificially discard this revenue so it doesn't pay down the national debt.
ENERGY	Stranded Asset Costs / Early Plant Retirements	Accelerated Capital Deprecation	delta_annual (Txl array)	Retiring power plants early means physical capital is destroyed before the end of its useful life, accelerating the rate at which K deprecates.	Capital Homogeneity: If OG-Core assumes 1 sector, Δ destroys all capital (laptops, restaurants). Must use multi_industry setup to isolate Δ to energy capital.
WATER	Unmet Water Demand / Drought Deficit (m^3)	Total Factor Productivity Shock	Z (TxM array)	Lack of water does not physically destroy tractors (K) or farmers (L). It simply makes them less efficient at producing agriculture output.	Elasticity Calibration: What is the elasticity of crops to water? We must derive exactly how a 10% water deficit translates to a X% drop in Z.
LAND	Agricultural Yield Penalty due to land degradation (Tons/Hectares)	Total factor productivity shock	Z (TxM array)	Similar to water, degraded land requires more capital/fertilizer to produce the same crop yield.	Confounding Shocks: If both Water and Land hit Z simultaneously, we risk double-counting the economic damage.
MACRO	GDP (Y_t) and Labor(L_t) Timepaths	Exogenous Energy/Water Demand	Specific AnnualDemand	As OG-Core shrinks or grows the economy due to these penalties, the physical demand for kilowatts and water changes proportionally.	The Price-Quantity Discrepancy: Requires iterative looping. If CLEWs lowers demand, the shadow price falls, meaning OG-Core must re-solve.

4.3 Modular Implementation Strategy

It is rather a theoretical economic challenge

With all the information that we have from research, I understand that integrating the bottom-up physical optimization framework (CLEWS) with a top-down macroeconomic general equilibrium model (OG-Core) is fundamentally a profound theoretical economic challenge, rather than a mere data engineering handoff.

A naive mapping of variables inevitably risks violating core macroeconomic accounting identities, such as artificially inflating government revenues when translating carbon shadow prices into consumption taxes, or triggering severe double-counting by simultaneously mapping energy transition costs into both aggregate Total Factor Productivity (TFP) shocks and increased uniform capital depreciation rates.

What the implementation strategy should look like

Therefore, our modular implementation strategy must begin with the simplest possible viable linkage: a minimalist, standalone proof-of-concept that establishes a single theoretically justified transmission channel (for instance, rigorously testing how a volumetric water deficit translates to a specific production function adjustment before attempting broader economic variables).

By initiating an iterative, 'soft-linking' convergence loop around these single points of interaction, we can observe the economic elasticities and ensure the models solve without breaking equilibrium. Only after mathematically validating that this foundational connection avoids the 'accounting traps' will we incrementally add complexity, carefully mapping and compounding additional physical constraints into the system step-by-step.

Current repository state and next steps

Currently the MUIOGO repository stands at a foundational baseline; it exists as a direct Python-and-Flask extraction of the legacy MUIO user interface, meaning the backend routes and the frontend components are tightly hard coupled to the bio-physical data parameters of OSeMOSYS. At this moment, there is zero macroeconomic architecture or OG-Core logic integrated into the codebase.

After careful assessments and discussions with the mentors, we can proceed by developing an isolated, standalone python sandbox entirely outside the primary web application. This prototype will start with iteratively executing a single, carefully mapped connection point, such as translating a CLEWS water constraint into a specific OG-Core production shock, to form an equilibrium seeking soft-link loop.

Once this foundational convergence logic is mathematically proven to avoid double counting and accounting traps, we will then refactor MUIOGO's monolithic backend into a decoupled orchestrator capable of managing distinct model runners and safely executing the validated translation layer.

---

5. The Preparation Repository

Github Repo: https://github.com/tejassinghbhati/ogclews-prep

Both the codebases are huge and need a lot of time to be understood. Therefore I have tried a different approach by running both the models locally and building a working data bridge between them. It took me some time to analyze and create this, and this preparation repository actually helped me build and observe a lot of things:

5.1 OG-Core Runner

```
outputs/
└── {run_id}/
    ├── metadata.json          # run config, status, timestamps
    ├── results/
    │   ├── OUTPUT_BASELINE/
    │   │   └── SS/
    │   │       └── SS_vars.pkl    # OG-Core steady-state solution
    │   └── summary.csv            # ETL-ready macro aggregates
    └── logs/
        └── run.log                # full solver output
```

To get a thorough understanding of the broader [MUIOGO](#) platform, I tried to create this first component. This standalone Python runner takes a YAML config file as input, runs OG-Core programmatically, captures all solver output to a structured log file. Then it writes output to a standardized directory tree with a run ID. Every run produces a self contained folder under output:

Link to the folder: https://github.com/tejassinghbhati/ogclews-prep/tree/main/01_og_runner

This is what the full outputs look like on running the model on sample data:

```
(ogclews) PS E:\MUIOGO Prep\01_og_runner> python run.py configs/example_config.yaml --no-tpi
[runner] Starting run: 20260321_104829_4f2d37
making dir:  outputs/20260321_104829_4f2d37/results\SS
making dir:  outputs/20260321_104829_4f2d37/results\TPI
In runner, baseline is  True
SS using initial guess factors for r and TR of 1.0 and 1.0 respectively.
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '-1.490e-08', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['2.977e-03', '1.150e-02', '-5.894e-02', '0.000e+00', '-4.347e-03', '1.324e-02', '1.829e-02', '1.984e-02', '1.212e-02', '1.443e-02', '1.749e-02', '3.488e-03', '-3.912e-04', '-3.462e-02']
GE loop errors = ['8.034e-04', '8.137e-04', '1.101e-03', '0.000e+00', '1.943e-03', '8.495e-06', '8.836e-06', '-5.712e-05', '2.756e-05', '5.463e-03', '5.392e-05', '-1.310e-03', '1.749e-04', '1.236e-04']
GE loop errors = ['1.275e-04', '1.307e-04', '-1.676e-04', '0.000e+00', '2.426e-05', '-3.606e-06', '-4.268e-06', '1.194e-06', '-1.705e-06', '5.300e-03', '-3.556e-06', '-1.326e-03', '2.183e-06', '-3.229e-06']
GE loop errors = ['-2.136e-04', '-2.190e-04', '2.992e-04', '0.000e+00', '-7.739e-05', '6.733e-06', '7.896e-06', '-1.857e-06', '3.467e-06', '5.308e-03', '6.250e-06', '-1.325e-03', '-6.965e-06', '7.230e-06']
GE loop errors = ['-8.041e-05', '-1.273e-04', '1.600e-03', '-2.236e-05', '4.618e-04', '-3.023e-05', '-2.291e-05', '-3.728e-05', '-2.932e-05', '4.221e-03', '-1.071e-05', '-1.062e-03', '4.156e-05', '5.704e-05']
GE loop errors = ['2.900e-04', '2.252e-04', '-6.339e-04', '1.709e-04', '4.499e-04', '1.783e-04', '1.826e-04', '1.647e-04', '1.695e-04', '2.902e-03', '1.826e-04', '-5.012e-04', '4.049e-05', '-1.599e-04']
GE loop errors = ['1.321e-04', '1.351e-04', '-5.409e-05', '0.000e+00', '1.238e-04', '-2.500e-06', '-1.318e-06', '-4.343e-06', '-1.041e-06', '-5.369e-04', '-2.834e-07', '1.355e-04', '1.114e-05', '-6.834e-06']
GE loop errors = ['1.791e-05', '1.831e-05', '-1.139e-05', '0.000e+00', '-1.990e-06', '-3.496e-07', '-3.549e-07', '-3.437e-07', '-4.948e-08', '-2.364e-04', '-2.090e-07', '5.887e-05', '-1.791e-07', '1.051e-06']
GE loop errors = ['-1.858e-05', '-1.898e-05', '8.129e-06', '0.000e+00', '3.494e-06', '2.700e-07', '3.131e-07', '3.241e-07', '-1.318e-07', '3.078e-04', '1.338e-07', '-7.695e-05', '3.145e-07', '-1.278e-06']
GE loop errors = ['1.937e-07', '1.972e-07', '3.866e-08', '0.000e+00', '-4.711e-07', '-3.496e-09', '-3.722e-09', '-4.408e-09', '1.788e-09', '-2.362e-06', '-1.754e-09', '5.770e-07', '-4.240e-08', '8.387e-08']
GE loop errors = ['5.525e-09', '5.724e-09', '-2.141e-08', '0.000e+00', '-7.663e-09', '-3.506e-10', '-4.359e-10', '7.140e-10', '-4.546e-10', '8.726e-08', '-5.935e-10', '-2.199e-08', '-6.896e-10', '-2.267e-09']
Iteration: 1  Distance: 3.690976263981321e-06
SS debt = 1.2067673364495197, 0.01534265672014735
IO: (1, 1), C: (1,)
Foreign debt holdings = 0.4827069345798079
Foreign capital holdings = 0.005164335705921
resource constraint: [-8.67951828e-10]
Steady state government spending is negative to satisfy budget
Checking constraints on capital, labor, and consumption.
There were no violations of the constraints on labor supply.
There were no violations of the constraints on consumption.
Maximum error in labor FOC = 1.4477308241112041e-13
Maximum error in savings FOC = 5.684341886008002e-14
C:\Users\HP\anaconda3\envs\ogclews\lib\site-packages\ogcore\SS.py:1612: UserWarning: Warning: The combination of the tax policy you specified and your target debt-to-GDP ratio results in an infeasible amount of government spending in order to close the budget (i.e.,
  warnings.warn(
JUST SAVED SS output to  outputs/20260321_104829_4f2d37/results\SS\SS_vars.pkl
It took 43.80150723473364 seconds to get that part done.
[runner] Completed: complete
[runner] Summary written to: outputs\20260321_104829_4f2d37\results\summary.csv
```

5.2 ETL Pipeline

There is supposed to be a pipeline that transforms outputs from the CLEWS/OsEMOSYS resource systems models into inputs for the OG-Core macroeconomic model, and vice-versa. Henceforth, as the second component I have created this declarative schema-driven data exchange pipeline which does the described job.

02_etl_pipeline > outputs > {} og_exchange.json > ...


```
{
  "generated_at": "2026-03-16T18:52:33.590355",
  "source_model": "CLEWS/OSeMOSYS",
  "target_model": "OG-Core",
  "metadata": {
    "data_dir": "data\\sample_clews_outputs",
    "schema": "exchange\\schema_clews_to_og.yaml"
  },
  "og_parameters": {
    "delta_tau_annual": 0.05,
    "tau_c": 0.01897625,
    "alpha_G": [
      0.055
    ]
  }
}
```


This pipeline is the intellectual bridge between two models that currently have no data connection. It is designed to be extended, which I have further planned to add a new variable mapping required editing the YAML file, not touching python code.

```
(ogclews) PS E:\MUIOGO Prep\02_etl_pipeline> python run_etl.py
[etl] Step 1: Validating inputs...
  All input files valid.
[etl] Step 2: Running transformations...
  [energy_investment_to_delta_tau] → delta_tau_annual = 0.05
  [co2_emissions_to_tau_c] → tau_c = 0.01897625
  [system_cost_to_alpha_G] → alpha_G = [0.055]
[etl] Step 3: Validating outputs...
  Validated params: {'delta_tau_annual': 0.05, 'tau_c': 0.01897625, 'alpha_G': [0.055
]}
[etl] Step 4: Writing exchange file...
[writer] Exchange file written to: outputs\og_exchange.json
[etl] Done.
```

This pipeline reads CLEWS output CSVs (capacity, emissions, system costs) organized by region, technology and year. Applies filters, aggregations, unit conversions and scaling transforms defined in a declarative YAML schema. The outputs look something like this:
Also on running the pipeline:

Link to the folder: https://github.com/tejassinghbhati/ogclews-prep/tree/main/02_etl_pipeline

5.3 The Validation Framework

I have created this framework for both OG-Core and CLEWS model inputs. It provides schema validation (via Pydantic), physical file-system checks, and formatted output suitable for terminals, REST APIs and log files.

This validation framework is the backend implementation of the requirement that the UI must have ‘informational error messages’ and ‘guardrails for non-technical users’. It returns structured errors in UI-consumable json format, so that we can allow even the policymakers to see what is wrong and not just the python traceback. The image below is the obtained terminal output on running the framework:

```
(.venv) (ogclews) PS E:\MUIOGO Prep\03_validation_framework> python run_validation.py
  --model og --config sample_data/valid_og_config.json
✓ Validation PASSED — all checks OK.
(.venv) (ogclews) PS E:\MUIOGO Prep\03_validation_framework> python run_validation.py
  --model og --config sample_data/invalid_og_config.json --output-format log
FAILED | errors=3 | warnings=0 | [schema] start_year: Field required | [schema] og_spec → maxiter: Input should be less than or equal to 1000 | [schema] og_spec → nu: Input should be less than or equal to 1
(.venv) (ogclews) PS E:\MUIOGO Prep\03_validation_framework> python run_validation.py
  --model og --config sample_data/invalid_og_config.json --output-format ui
>>
{
  "passed": false,
  "error_count": 3,
  "errors": [
    {
      "field": "start_year",
      "message": "Field required",
      "expected": "see message",
      "received": "{'scenario_name': 'broken_config', 'country_module': 'og_usa.calibrate', 'time_path': False, 'og_spec': {'maxiter': 5000, 'nu': 1.5, 'mindist_SS': 1e-09}}"
    },
    {
      "field": "og_spec \u2192 maxiter",
      "message": "Input should be less than or equal to 1000",
      "expected": "see message",
      "received": "5000"
    },
    {
      "field": "og_spec \u2192 nu",
      "message": "Input should be less than or equal to 1",
      "expected": "see message",
      "received": "1.5"
    }
  ],
  "warnings": []
}
(.venv) (ogclews) PS E:\MUIOGO Prep\03_validation_framework>
```

Link to the folder:
https://github.com/tejassinghbhati/ogclews-prep/tree/main/03_validation_framework

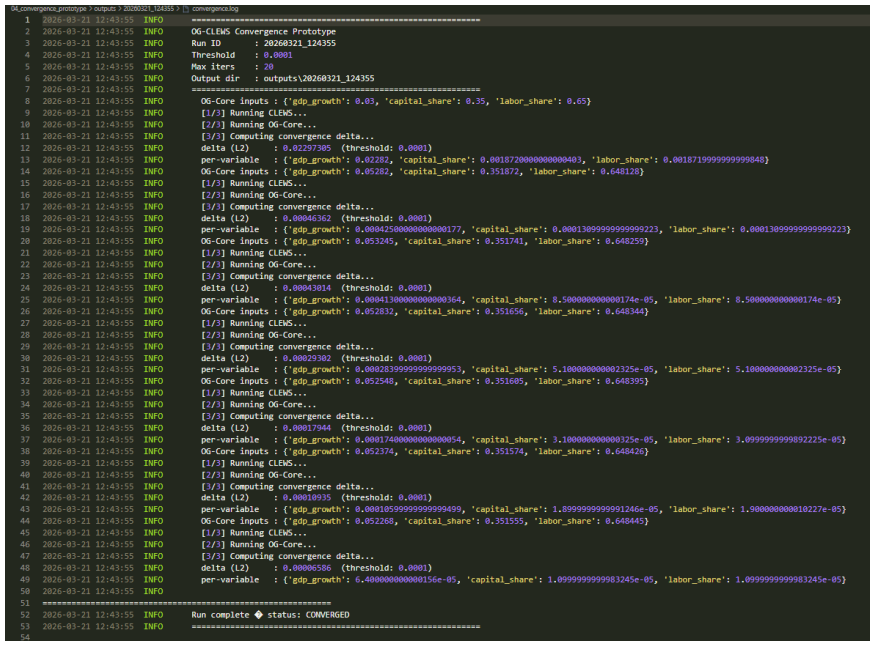
5.4 Convergence Prototype - OG CLEWS

The hardest architectural challenge of the full project was to make the two independent models talk to each other until their outputs stabilize into a consistent solution. Therefore there was a need to develop a convergence loop between the CLEWS/OSeMOSYS resource systems model and the OG-Core macroeconomic model.

Mode	Description
Coupled (one-way)	CLEWS → ETL → OG-Core, single pass
Coupled (two-way)	CLEWS → OG-Core → CLEWS, single feedback loop
Converging	**CLEWS ↔ OG-Core iterated until stable** ← this prototype

This prototype runs CLEWS (mocked) with current macroeconomic parameters, transforms its outputs into OG-Core inputs by declarative mapping, then runs OG-Core with those transformed inputs, computes a L2 norm convergence delta between successive parameter vectors, stops when delta falls below threshold or max iterations have reached. Then it logs every iteration's state to a structured file.

The screenshot I have attached is of the convergence.log file (demonstrating full iteration log) generated on running the project. Apart from this another files history.json and metadata.json get generated respectively demonstrating per iteration state and run summary. Right now this prototype only implements one way coupling with feedback (CLEWS → OG-Core → CLEWS). The full GSoC project extends this to:



```
04_convergence_prototype > outputs > 20260321_124355 > convergence.log
2026-03-21 12:43:55  INFO    ============================================================
2026-03-21 12:43:55  INFO    OG-CLEWS Convergence Prototype
2026-03-21 12:43:55  INFO    Run ID       : 20260321_124355
2026-03-21 12:43:55  INFO    Threshold    : 0.0001
2026-03-21 12:43:55  INFO    Max iters    : 20
2026-03-21 12:43:55  INFO    Output dir   : outputs\20260321_124355
2026-03-21 12:43:55  INFO    ============================================================
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.03, 'capital_share': 0.35, 'labor_share': 0.65}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.02297305  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 0.02282, 'capital_share': 0.001872000000000403, 'labor_share': 0.0018719999999999848}
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.05282, 'capital_share': 0.351872, 'labor_share': 0.648128}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.00046362  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 0.000425000000000177, 'capital_share': 0.000130999999999223, 'labor_share': 0.000130999999999223}
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.053245, 'capital_share': 0.351741, 'labor_share': 0.648259}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.00043014  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 0.000413000000000364, 'capital_share': 8.500000000000174e-05, 'labor_share': 8.500000000000174e-05}
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.052832, 'capital_share': 0.351656, 'labor_share': 0.648344}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.00029302  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 0.000283999999999953, 'capital_share': 5.100000000002325e-05, 'labor_share': 5.100000000002325e-05}
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.052548, 'capital_share': 0.351605, 'labor_share': 0.648395}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.00017944  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 0.000174000000000054, 'capital_share': 3.100000000000325e-05, 'labor_share': 3.099999999892225e-05}
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.052374, 'capital_share': 0.351574, 'labor_share': 0.648426}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.00010935  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 0.000105999999999499, 'capital_share': 1.899999999991246e-05, 'labor_share': 1.900000000010227e-05}
2026-03-21 12:43:55  INFO      OG-Core inputs : {'gdp_growth': 0.052268, 'capital_share': 0.351555, 'labor_share': 0.648445}
2026-03-21 12:43:55  INFO      [1/3] Running CLEWS...
2026-03-21 12:43:55  INFO      [2/3] Running OG-Core...
2026-03-21 12:43:55  INFO      [3/3] Computing convergence delta...
2026-03-21 12:43:55  INFO      delta (L2)     : 0.00006586  (threshold: 0.0001)
2026-03-21 12:43:55  INFO      per-variable   : {'gdp_growth': 6.400000000000156e-05, 'capital_share': 1.099999999983245e-05, 'labor_share': 1.099999999983245e-05}
2026-03-21 12:43:55  INFO
============================================================
2026-03-21 12:43:55  INFO    Run complete ◆ status: CONVERGED
2026-03-21 12:43:55  INFO    ============================================================
```

Link to the folder:

https://github.com/tejassinghbhati/ogclews-prep/tree/main/04_convergence_prototype

5.5 OG-Core Flask API & Country Scenario

To validate the full integration path, I extended MUIOGO's Flask backend with five OG-Core endpoints mirroring the existing Blueprint pattern which is to run dispatch, status polling, results retrieval, run listing, and deletion. Also I traced the complete workflow end-to-end on a real country, modelling Mauritius's renewable energy transition through CLEWS and into OG-Core, producing macroeconomic results that show a +0.26pp GDP growth lift and +2.4pp fiscal revenue gain over the baseline.
Link to OG-Core Flask API folder:
https://github.com/tejassinghbhati/ogclews-prep/tree/main/05_ogcore_api
Link to Mauritius Country Scenario:
https://github.com/tejassinghbhati/ogclews-prep/tree/main/06_country_scenario

---

6. Timeline

Pre-GSoC (Now → April 29)

I did not want to show up to this programme having only read documentation. The preparation repository documents everything I have built — an OG-Core runner, a bidirectional ETL pipeline, a validation framework, a convergence prototype, Flask API endpoints for MUIOGO,

and an end-to-end Mauritius country scenario. Between now and April 29, I will continue raising issues on the MUIOGO repository and deepening my study of both models' theoretical foundations in preparation for the framework assessment phase.

Community Bonding (April 30 – May 25)
The Goal of this period is to design documentation, get reviews from mentors and prepare on how effectively implementation can be brought.

Conduct a deep study of OG-Core's objective functions, household optimization problem, and TPI solver structure, specifically to identify every parameter that can be legitimately exogenized without breaking the equilibrium

- Conduct a parallel study of CLEWS's constraint structure, specifically to map which output variables are physically meaningful as economic signals and which are not
- Produce a written Framework Assessment Document covering: identified hard stops where the models mathematically contradict each other, proposed transmission channels for each CLEWS output variable, theoretical justification for each channel, and risks of double-counting or accounting identity violations
- Post this document as a tracked issue on the MUIOGO repository for mentor review and feedback before Week 1
- Validate the full integration pipeline runs cleanly across Windows, Linux, and macOS — building on cross-platform work from merged PRs #34, #35, #43, #44
- Align with mentors on the primary country dataset — OG-ZAF, OG-PHL, or another configuration — that will serve as the demonstration target

Week 1 (May 26 – June 1)
Framework Validation and Schema Design: The Framework Assessment Document from community bonding gets reviewed with mentors and finalized. Nothing is implemented before this is agreed.

- Present the framework document and incorporate mentor feedback
- Finalize the translation matrix, every CLEWS output variable mapped to its OG-Core transmission channel, with agreed theoretical justification and acceptable distortion bounds
- Define the og_exchange.json schema based on the agreed translation matrix - not before.
- Open a tracked issue for each planned GSoC deliverable with explicit acceptance criteria derived from the agreed framework
- Study Contributor 2's frontend design decisions so backend output schemas are integration-ready from Week 2

Deliverable: agreed framework document, finalized translation matrix, stable og_exchange.json schema

Week 2 (June 2 – June 8)
Module - OG-Core Runner — production-grade: With the framework agreed, implementation begins. The og_runner proof of concept from the preparation repository gets hardened into a production-ready component.

- Implement pydantic-validated RunConfig dataclass for YAML config ingestion
- Add structured logging of all solver inputs and outputs to a timestamped run directory
- Add clean error handling for the UserWarning infeasibility case encountered during local testing, where a tax policy + debt-to-GDP combination produces $G < 0$
- Write pytest coverage for config loading, execution paths, and output file verification

Deliverable: tested, documented OG-Core runner module with structured log output and clean error handling

Week 3 (June 9 – June 15)
Module - CLEWS Runner Wrapper: The symmetric counterpart to the OG-Core runner. Both models need consistent entry points for the coupled orchestrator.

- Build a wrapper that invokes a CLEWS/OSeMOSYS solve programmatically and captures output CSVs, capacity, emissions, system costs, water use
- Structure captured outputs into a consistent intermediate format that mirrors the OG-Core runner's output directory structure
- Validate the wrapper runs correctly across all three platforms
- Write pytest coverage for invocation, output capture, and intermediate format validation

Deliverable: CLEWS runner wrapper that produces structured output ready for ETL ingestion

Week 4 (June 16 – June 22)
Module - Bidirectional ETL Pipeline: The preparation repository demonstrated one mapping. This week implements all agreed mappings from the translation matrix and adds the reverse direction.

- Extend the ETL pipeline to cover all CLEWS-to-OG-Core parameter mappings from the agreed translation matrix
- Implement the reverse direction — OG-Core outputs back to CLEWS inputs — to support iterative coupling
- Add pydantic validation for every transformation step
- Write pytest tests for every mapping in both directions

Deliverable: complete, bidirectional ETL pipeline with full test coverage and validated parameter outputs

Week 5 (June 23 – June 29)

Module - OG–CLEWS Coupled Module — first end-to-end run: This is the week all three components connect for the first time. Everything before this was either framework or preparation.

- Connect CLEWS runner → ETL transformer → og_exchange.json → OG-Core runner into a single callable pipeline
- Run the pipeline end-to-end on sample data and verify OG-Core produces a valid steady-state result from CLEWS-derived inputs without triggering the infeasibility warning
- Verify that the agreed translation channels do not produce double-counting or accounting identity violations — this is the practical test of the framework
- Document all intermediate file locations and output structure for Contributor 2's frontend integration

Deliverable: first working end-to-end coupled integration on sample data, validated against framework constraints

Week 6 (June 30 – July 6)
Module - Coupled Module — real country dataset: Sample data is a proof of concept. A real country dataset is a proof of value.

- Replace sample data with the real country dataset agreed during community bonding
- Run the full coupled pipeline end-to-end and document all observations — which translation channels held, which required adjustment, and what OG-Core's output looks like compared to the standalone baseline
- Produce the "at least one end-to-end workflow demonstrated" deliverable required by the midterm milestone

Deliverable: documented country run with comparison against OG-Core standalone baseline

Week 7 (July 7 – July 13)
Output Validator and midterm preparation: The coupled pipeline produces results. The output validator answers whether those results are trustworthy.

- Implement an output validator that compares the integrated pipeline's OG-Core results against a standalone baseline
- Flag statistically significant deviations in key macroeconomic aggregates — GDP, wages, interest rates, tax revenues
- Produce a structured diff report
- Document all risks identified through Weeks 1–6 with mitigation plans

Deliverable: output validator with diff report; risk register posted in issues; all midterm criteria met

Week 8 — Midterm Evaluation (July 14 – July 20)
Milestone review - framework agreed, core implementation reviewable, end-to-end workflow demonstrated: At midterm the following will be complete and reviewable:

- Agreed framework document and translation matrix
- OG-Core runner (production-grade, tested, logged)
- CLEWS runner wrapper (cross-platform, tested)
- Bidirectional ETL pipeline (full coverage, pydantic-validated)
- og_exchange.json exchange schema (versioned, documented)
- First country run with output comparison against standalone baseline
- Documented risk register

Week 9 (July 21 – July 27)

Module - OG–CLEWS Converging Module: The converging module is the hardest deliverable of the entire project and also the one I have thought about the most, the convergence prototype in the preparation repository exists specifically so I could understand this problem before claiming I could solve it.

- Expand the coupled module into iterative execution: run CLEWS, feed outputs into OG-Core, feed OG-Core outputs back into CLEWS, repeat until results stabilise
- Implement per-variable convergence detection — each key variable has its own configured tolerance threshold, not a single aggregate metric
- Add full iteration logging so every pass is traceable and debuggable
- Write pytest coverage for convergence behaviour and loop termination

Deliverable: working converging module with per-variable convergence detection and iteration logging

Week 10 (July 28 – August 3)

Cross-platform verification and CI: Cross-platform issues are not theoretical for this codebase — I have fixed hardcoded path separators, dead solver stubs, and platform-specific file handle behaviour across PRs #34, #35, #43, and #44. This week closes the remaining gaps at the pipeline level.

- Systematically verify the full pipeline runs without modification on Windows, Linux, and macOS
- Produce a CI-ready setup verification script and cross-platform test matrix

Deliverable: cross-platform test matrix; platform-independence verification script

Week 11 (August 4 – August 10)
Documentation and example notebook: The pipeline is only useful if a country team can run it without asking for help.

- Write complete user-facing documentation: installation guide, configuration reference, worked example
- Build a Jupyter notebook demonstrating the full integration for the country dataset from Week 6 — this is the primary handoff artifact for UN country teams after GSoC ends

Deliverable: user guide, configuration reference, worked example, reproducible Jupyter notebook

Week 12 (August 11 – August 17)
Final closure and reproducibility check

- Close every scoped issue or explicitly defer it with documented rationale
- Conduct a full end-to-end reproducibility check: fresh clone, follow documentation, verify on all three platforms
- Address final mentor review feedback
- Confirm backend output schemas are fully documented for Contributor 2

Deliverable: all scoped issues closed or deferred; reproducibility verified from fresh clone on all three platforms

Week 13 — Buffer Week (August 18 – August 23)
Realistically, something will slip. This week exists to absorb that. If nothing slips, the stretch goals are: second country dataset demonstration, contributing platform-independence improvements upstream to OSeMOSYS/MUIO, exploratory work on converging module performance.

Final evaluation submission: August 24

---

7. Technical Journey and Previous Work

A lot of my technical journey has been driven by the urge to make complex systems usable by people who actually need them. And this curiosity has led me to work on very diverse tech stacks from scientific systems, automations, machine learning, blockchain to open source, which ultimately led me here applying at GSoC.

Numerical Modeling and Simulation Systems: have worked on various projects learning model integrations, designing robust schemas and creating natural resource models. Links: https://github.com/tejassinghbhati/CSALABS https://github.com/tejassinghbhati/ogclews-prep
LLMs and AI/RAG Systems: Built offline knowledge apps using Ollama, ChromaDB vector store, sentence-transformers, and full RAG Pipelines running locally with zero cloud dependency. Links: https://github.com/tejassinghbhati/MindVault, https://github.com/tejassinghbhati/Local-LLM-Practice
Machine Learning and Deep Learning: Implementation of Neural Style Networks using Pytorch + TorchVision, processing real-time webcam feeds through trained Deep Learning Model served via FastAPI, built real time computer vision pipelines for background subtraction and various object detections. Links: https://github.com/tejassinghbhati/bakashi_studio, https://github.com/tejassinghbhati/OpenCV-projects
Full Stack Development: Experienced using MERN stack, Nextjs, editors, FastAPI, ChromaDB, Docker and various tools for many development projects across college and self learning journey.
Open Source Contributions: Engaged with IntelOwl: a 4.5 star Python cybersecurity platform for threat intelligence and management at scale, Neural-LAM: an active research ML project, working with xarray, zarr, W&B, MLFlow, and distributed training pipelines, OmegaUp: a 342-star competitive programming contest evaluator built in PHP, used across Latin America for programming education.
Hackathons: Represented India at WCHL 2025, one of the world's largest Web3 hackathons, presenting DAOverse on a global stage. Won a college hackathon building Vault — a blockchain-based document verification platform using Ethereum Smart Contracts and IPFS, in 24 hours.

8. A Note from the Author

Lack of financial resources might not be a huge problem of our time, yet the lack of natural resources definitely is. I feel like the last generation has exploited the resources in many ways and it is certainly the duty of our generation to make things right. Growing up I never knew a world without the looming crisis of climate change. I have seen it through my eyes in small villages across deserts in India, the water scarcity, energy crisis and land degradation has such a drastic effect on people’s lives.

I was fortunate to grow up in a family and at a place where I could afford the time to focus on my studies, read and learn so many things, sometimes I imagine the fate of a child born in a small poor village in Rajasthan, at the middle of the scorching desert, he would have to work in the fields from a very young age, he would never have the opportunity to gain knowledge and learn skills to get better at literally anything. The parents lack awareness to such an extent that children are married off early at small ages, live their lives in poverty in a scarce region where even the basic necessity is considered a luxury.

These people know the value of life more than us, they know how to survive in the toughest conditions, there would not be a bigger opportunity for me to contribute something, to a project that is building an integration between models that end up informing decisions that reach people who will never know my name. The people who understand things more valuable than data on spreadsheets or data modelled in python files.

I cannot think of a more honest reason to spend my summer writing code.

Thank you so much for going through my proposal and giving it a consideration. I genuinely hope to get a chance to work on this.